BCA Semester 1 Examination 2025

Model Solutions: 1BCA4 - Programming in C

Session	Paper Code	Time	Full Marks	Pass Marks
2024-2027	111604	3 Hours	100	40

Instructions: Answer any five questions (equal marks). Solutions for all questions are given below for practice. All programs were compiled with GCC. Write answers in your own words in the exam.

Q1. Define Array. Write a program in C to find the sum of odd and even numbers in a given array.

Array

An array is a collection of elements of the **same data type** stored in **contiguous memory locations** and accessed by a common name and an index (subscript). The index starts at 0. Declaration: `int marks[5];` (elements marks[0] to marks[4]). Initialisation: `int marks[5] = {70, 65, 80, 90, 55};` Types: one-dimensional, two-dimensional (matrix) and multi-dimensional.

Program

```
#include <stdio.h>
int main()
{
    int a[50], n, i, even_sum = 0, odd_sum = 0;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter %d numbers: ", n);
    for (i = 0; i < n; i++)
    {
        scanf("%d", &a[i]);
        if (a[i] % 2 == 0)
            even_sum += a[i];
        else
            odd_sum += a[i];
    }
    printf("Sum of even numbers = %d\n", even_sum);
    printf("Sum of odd numbers  = %d\n", odd_sum);
    return 0;
}
```

```
Enter number of elements: 5
Enter 5 numbers: 10 15 20 25 30
Sum of even numbers = 60
Sum of odd numbers  = 40
```

A number is even if the remainder on dividing by 2 is zero (a[i] % 2 == 0); otherwise it is odd.

Q2. What is an operator? Discuss various types of operators, also discuss precedence and associativity of operators.

An operator is a symbol that tells the compiler to perform a specific mathematical, relational or logical operation on operands. Expression example: a + b (+ is the operator, a and b are operands).

Types of operators

Type	Operators	Example
Arithmetic	+ - * / %	a % b gives the remainder
Relational	< > <= >= == !=	a > b gives 1 or 0
Logical	&& || !	(a > 5) && (b < 5)
Assignment	= += -= *= /= %=	a += 3 means a = a + 3
Increment / decrement	++ --	i++ (post), ++i (pre)
Bitwise	& | ^ ~ << >>	a & b, a << 1
Conditional (ternary)	? :	max = (a > b) ? a : b;
Special	sizeof, comma (,), & address, * pointer	sizeof(int)

Precedence and associativity

Precedence decides which operator is evaluated first when an expression has several operators (e.g. * before +). **Associativity** decides the direction of evaluation (left to right or right to left) when operators have the same precedence. Parentheses ( ) override both.

Priority (high to low)	Operators	Associativity
1	() [] . ->	Left to right
2	! ~ ++ -- unary + - * & sizeof	Right to left
3	* / %	Left to right
4	+ -	Left to right
5	<< >>	Left to right
6	< <= > >=	Left to right
7	== !=	Left to right
8	& ^ | (bitwise)	Left to right
9	&& then ||	Left to right
10	?:	Right to left
11	= += -= *= /= %=	Right to left
12	, (comma)	Left to right

Example: 2 + 3 * 4 = 14 (since * has higher precedence); a = b = 5 is evaluated right to left (b = 5 first, then a = b).

```
#include <stdio.h>
int main()
{
    int a = 7, b = 2;
    printf("%d %d %d %d %d\n", a + b, a - b, a * b, a / b, a % b);   /* 9 5 14 3 1 */
    printf("%d %d\n", a > b, a == b);                                 /* 1 0 */
    printf("%d\n", (a > 5) && (b < 5));                               /* 1 */
    a += 3;                                                           /* a = 10 */
    printf("%d\n", a);
    printf("%d\n", (a > b) ? a : b);                                  /* 10 */
    printf("%d\n", 2 + 3 * 4);                                        /* 14 */
    return 0;
}
```

Q3. Write a program in C to calculate the length of a string without using library function.

A string ends with the null character '\0'. We count characters one by one with a loop until '\0' is reached (fgets also stores the newline '\n', which is excluded from the count).

```
#include <stdio.h>
int main()
{
    char str[100];
    int i = 0, length = 0;
    printf("Enter a string: ");
    fgets(str, sizeof(str), stdin);
    while (str[i] != '\0' && str[i] != '\n')
    {
        length++;
        i++;
    }
    printf("Length of the string = %d\n", length);
    return 0;
}
```

```
Enter a string: Hello World
Length of the string = 11
```

Q4. Explain storage class modifiers with all its features. Also give suitable examples.

A storage class tells the compiler four things about a variable: its **storage place** (memory or register), **default initial value**, **scope** (where it is visible) and **lifetime** (how long it exists).

Storage class	Storage	Default value	Scope	Lifetime
auto	Memory (stack)	Garbage	Local to block	Till the block ends
register	CPU register (if available)	Garbage	Local to block	Till the block ends
static	Memory (data segment)	0	Local to block (or file if global)	Whole program; value retained
extern	Memory (data segment)	0	Global (all files)	Whole program

- **auto:** default class of local variables; `auto int x;` is the same as `int x;`.
- **register:** faster access for heavily used variables (loop counters); address operator (&) cannot be applied.
- **static:** initialised only once; retains its value between function calls.
- **extern:** declares a global variable defined elsewhere (another file or later in the file) without allocating new memory.

```
#include <stdio.h>
int g = 100;                /* global variable, defined here */
void show()
{
    extern int g;           /* extern: refers to the global g */
    printf("g = %d\n", g);
}
void count()
{
```

```
    static int c = 0;        /* static: keeps its value between calls */
    int a = 0;               /* auto: created again on every call */
    c++; a++;
    printf("static c = %d, auto a = %d\n", c, a);
}
int main()
{
    register int i;          /* register: request to keep i in a CPU register */
    for (i = 0; i < 3; i++)
        count();
    show();
    return 0;
}
```

```
static c = 1, auto a = 1
static c = 2, auto a = 1
static c = 3, auto a = 1
g = 100
```

Q5. What is format specifier? Write the uses of different format specifiers with example.

A format specifier is a code beginning with **%** used in printf() and scanf() to tell the compiler the data type of the value to be printed or read.

Specifier	Use	Data type
%d or %i	Signed decimal integer	int
%u	Unsigned decimal integer	unsigned int
%ld / %lu	Long integer	long / unsigned long
%f	Floating-point number (6 decimals by default)	float, double (printf)
%lf	Double (used in scanf)	double
%e / %E	Scientific notation	float, double
%c	Single character	char
%s	String	char array
%o	Octal number	int
%x / %X	Hexadecimal number (lower / upper case)	int
%p	Address (pointer)	pointer
%%	Prints the % sign	-

Width and precision can be given: **%5d** (minimum width 5), **%-5d** (left aligned), **%.2f** (2 digits after the decimal point).

```
#include <stdio.h>
int main()
{
    int a = 25;  float f = 3.14159;  double d = 12.3456789;  char ch = 'A';
    char name[] = "Ravi";  unsigned int u = 40000;  long l = 1234567L;
    printf("%d\n", a);        /* 25 */
    printf("%f\n", f);        /* 3.141590 */
    printf("%.2f\n", f);      /* 3.14 */
    printf("%lf\n", d);       /* 12.345679 */
    printf("%c\n", ch);       /* A */
    printf("%s\n", name);     /* Ravi */
    printf("%u\n", u);        /* 40000 */
```

```
    printf("%ld\n", l);       /* 1234567 */
    printf("%o %x %X\n", a, a, 255);   /* 31 19 FF */
    printf("%e\n", d);        /* 1.234568e+01 */
    printf("%5d|%-5d|\n", a, a);       /*    25|25   | */
    return 0;
}
```

Q6. Write a program in C to check whether inputted number is perfect or not.

A perfect number is a number equal to the sum of its proper divisors (divisors excluding the number itself). Example: 6 = 1 + 2 + 3, 28 = 1 + 2 + 4 + 7 + 14.

```
#include <stdio.h>
int main()
{
    int n, i, sum = 0;
    printf("Enter a number: ");
    scanf("%d", &n);
    for (i = 1; i <= n / 2; i++)
    {
        if (n % i == 0)
            sum += i;          /* add all proper divisors */
    }
    if (n > 0 && sum == n)
        printf("%d is a perfect number\n", n);
    else
        printf("%d is not a perfect number\n", n);
    return 0;
}
```

```
Enter a number: 28
28 is a perfect number

Enter a number: 12
12 is not a perfect number   (1+2+3+4+6 = 16)
```

Q7. What is selection control structure? Discuss in detail with example.

A selection (decision-making) control structure lets the program choose which statements to execute depending on whether a condition is true or false. C provides:

Statement	Syntax / use
Simple if	if (condition) { statements; } - executes only when the condition is true.
if-else	if (condition) { A } else { B } - one of two blocks.
Nested if	An if or if-else placed inside another if/else for dependent conditions.
else-if ladder	if (c1) {..} else if (c2) {..} else if (c3) {..} else {..} - first true condition is executed.
switch-case	switch (expression) { case value: ... break; default: ... } - multi-way choice on an integer/char value.
Conditional operator	(condition) ? value1 : value2

```
if (a > b)
    printf("a is greater");
else
    printf("b is greater or equal");
```

Program using else-if ladder and switch

```
#include <stdio.h>
```

```
int main()
{
    int marks, day;
    printf("Enter marks: ");
    scanf("%d", &marks);
    if (marks >= 75)
        printf("Distinction\n");
    else if (marks >= 60)
        printf("First class\n");
    else if (marks >= 40)
        printf("Pass\n");
    else
        printf("Fail\n");

    printf("Enter day number (1-3): ");
    scanf("%d", &day);
    switch (day)
    {
        case 1: printf("Monday\n");  break;
        case 2: printf("Tuesday\n"); break;
        case 3: printf("Wednesday\n"); break;
        default: printf("Invalid day\n");
    }
    return 0;
}
```

Any non-zero value is true and zero is false in C. Braces are needed when a branch has more than one statement.

Q8. Differentiate between the following: (i) while and do-while loop (ii) break and continue (iii) automatic and static variables (iv) getch() and getche()

(i) while and do-while loop

while loop	do-while loop
Entry-controlled: condition is tested before the body.	Exit-controlled: condition is tested after the body.
Body may not execute even once.	Body executes at least once.
No semicolon after while(condition).	Semicolon is required after while(condition);
while (i <= 5) { ... }	do { ... } while (i <= 5);

(ii) break and continue

break	continue
Terminates the loop or switch completely.	Skips the rest of the current iteration only.
Control goes to the statement after the loop.	Control goes to the next iteration (condition/increment).
Used in loops and switch.	Used only in loops.

(iii) Automatic and static variables

Automatic (auto) variable	Static variable
Default storage class of local variables.	Declared with the keyword static.
Initial value is garbage.	Initial value is zero.
Created each time the function is called and destroyed on exit.	Created once and retains its value until the program ends.

Automatic (auto) variable	Static variable
Stored on the stack.	Stored in the data segment.

(iv) getch() and getche()

getch()	getche()
Reads a single character from the keyboard without displaying (echoing) it on the screen.	Reads a single character and echoes (displays) it on the screen.
Does not wait for Enter key.	Does not wait for Enter key.
Used for passwords and 'press any key'.	Used when the typed character should be visible.
Both are declared in conio.h (non-standard, Turbo C / Windows).	Same header file.

Q9. What is an operator? Discuss relational and logical operator with example. Also discuss unary and binary operator.

An operator is a symbol that performs an operation on operands (see Q2 for all types).

Relational operators

Compare two values and return 1 (true) or 0 (false).

Operator	Meaning	Example (a = 10, b = 20)	Result
<	Less than	a < b	1
>	Greater than	a > b	0
<=	Less than or equal to	a <= 10	1
>=	Greater than or equal to	a >= b	0
==	Equal to	a == b	0
!=	Not equal to	a != b	1

Logical operators

Operator	Meaning	Example	Result
&&	AND - true if both conditions are true	(a < b) && (b > 15)	1
||	OR - true if at least one is true	(a > b) || (b > 15)	1
!	NOT - reverses the result	!(a < b)	0

Unary and binary operators

- **Unary:** need one operand: ++, --, unary minus (-a), !, ~, sizeof, & (address of), * (pointer).
- **Binary:** need two operands: + - * / %, relational, logical (&&, ||), assignment (=), bitwise (&, |, ^).
- **Ternary:** three operands: ?: (only one in C).

```
#include <stdio.h>
int main()
{
    int a = 10, b = 20;
    printf("%d %d %d\n", a < b, a == b, a != b);          /* 1 0 1 */
    printf("%d\n", (a < b) && (b > 15));                  /* 1 */
```

```
    printf("%d\n", (a > b) || (b > 15));          /* 1 */
    printf("%d\n", !(a < b));                     /* 0 */
    printf("%d\n", -a);                           /* unary minus: -10 */
    a++;                                          /* unary increment: a = 11 */
    printf("%d\n", a + b);                        /* binary plus: 31 */
    return 0;
}
```

Q10. Write short notes on any two of the following with example: (i) Continue (ii) Identifier (iii) Escape sequence character

(i) continue

The continue statement skips the remaining statements of the current iteration of a loop and transfers control to the next iteration (to the increment part in a for loop, or to the condition test in while and do-while). It does not end the loop.

```
#include <stdio.h>
int main()
{
    int i;
    for (i = 1; i <= 10; i++)
    {
        if (i % 3 == 0)
            continue;          /* skip multiples of 3 */
        printf("%d ", i);
    }
    printf("\n");              /* 1 2 4 5 7 8 10 */
    return 0;
}
```

(ii) Identifier

An identifier is the name given by the programmer to a variable, function, array, structure etc. **Rules:** only letters, digits and underscore are allowed; it must not start with a digit; no spaces or other special characters; keywords such as int, if, while cannot be used; C is case sensitive (Sum and sum differ). Valid: `total, _count, student_name, marks1`. Invalid: `1st, my-name, float`.

(iii) Escape sequence characters

Escape sequences begin with a backslash and represent special characters that cannot be typed directly in a string.

Sequence	Meaning	Sequence	Meaning
\n	New line	\\	Backslash
\t	Horizontal tab	\"	Double quote
\a	Alert (beep)	\'	Single quote
\b	Backspace	\r	Carriage return
\0	Null character	\f	Form feed

Example: `printf("Name:\tRavi\nCity:\tGaya\n");` prints two lines with a tab gap after the labels.